Week 8: Language models and the transformer

Jin-Guo Liu, Xi-Wei Pan

Abstract: We learn how a language model assigns probabilities to text and generates new text. We look inside a pretrained table that turns tokens into vectors and find relations between words stored as directions. We predict each token of short children's stories from the ones before it, first by counting, which the curse of dimensionality limits to short contexts, then with an MLP. We derive the MLP as a memory of key–value facts written into its weights, and measure how many facts such a memory holds. Facts stated in the text itself cannot be written into fixed weights, so we derive attention as a memory built from the context. We assemble the two blocks into a transformer, train it on the stories and sample new ones. Finally, we refit the Chinchilla scaling law to published training runs and find how to split a compute budget between model size and data. The Goal lines state the numerical targets.

Language models

Text as a sequence of symbols. Week 7 generated images, each a vector of a fixed number of real values. A text is a string of characters of any length. A *language model* assigns a probability to every string and can draw new ones. We write a string as a sequence $x_1, x_2, \ldots, x_T$ of symbols from a finite set, the tokens below, and predict each one from the ones before it. That single idea, applied with better and better predictors, is the whole week. Figure 1 shows the machine the week builds, and which Part builds each piece.

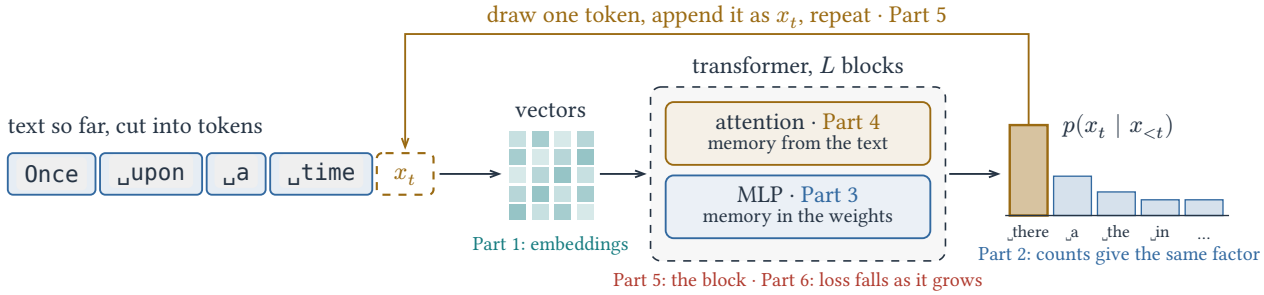


Figure 1: A language model at work. The text so far is cut into tokens, each token becomes a vector, and a stack of transformer blocks turns the vectors into a distribution for the next token. One token is drawn from it and appended, and the loop repeats. Coloured tags name the Part that builds or studies each piece.

Tokens and the tokenizer. A token is the basic unit a model reads and writes: an integer standing for a piece of text. The *vocabulary* is the set of all V tokens. A *tokenizer* is trained once on a fixed *corpus*, a collection of texts, which fixes its vocabulary. It then *encodes* any string into tokens and *decodes* tokens back into the string. A byte-pair encoding starts from single characters and repeatedly merges the most frequent adjacent pair into a new token until the vocabulary is full, so frequent pieces such as " the" or " time" become single tokens. We download one that Karpathy trained on TinyStories, with $V = 512$ tokens. Three are special, among them the *end-of-sequence* token that separates stories. Another 256 stand for single bytes, a fallback that lets any text be encoded. The remaining 253 are merged pieces of text.

The normalizer of a language model. Week 6's energy models needed a normalizer Z, a sum over all configurations that we could not compute. A language model avoids it: each prediction is a probability distribution over a finite set, normalized by a sum we can compute. Parts 2 to 5 rely on this.

The path through the week. Figure 2 shows the six Parts as one chain. Part 1 turns tokens into vectors a model can read. Parts 2 and 3 predict the next token, first by counting and then with an MLP, which stores facts in its weights. A fact stated only in the current text never reaches those weights. Part 4 builds a second memory from the context itself, attention. Part 5 combines the two blocks into a transformer, trains it on the stories, and samples from it. Part 6 asks how the loss improves with model size and data, and how to spend a compute budget.

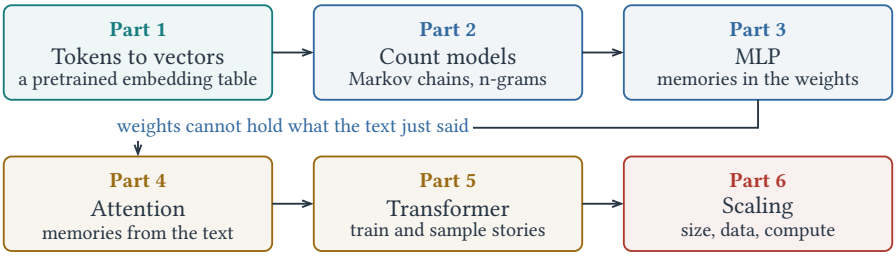


Figure 2: The week's Parts. Each model answers a limit of the one before it; the arrow between the rows marks the limit that motivates attention.

Resources:

- Andrej Karpathy, *Let's build GPT: from scratch, in code, spelled out*. Watch for the character model, self-attention and the training loop of Parts 2 to 5.
- 3Blue1Brown, *Attention in transformers, visually explained*. Watch for queries, keys and values, used in Parts 4 and 5.
- Alex Wang, *You could have invented transformers from memory*. Read how the outer-product store, the MLP and self-attention each store key–value memories, for Parts 3 and 4.
- Hazy Research, *Zoology: measuring and improving recall in efficient language models*. Read why recall from context is where attention wins, for Part 4.
- Lilian Weng, *Scaling laws, carefully*. Read the sections on fitting in practice, for Part 6.

Part 1: From tokens to vectors: embeddings

Goal: look inside a small pretrained embedding table and show that meaning becomes geometry: a relation between words, such as country to capital, is one direction. For three relations of nine word pairs each, a pair's offset has a mean cosine of at least 0.3 with the average offset of the other pairs, while a copy of the table with shuffled rows stays within ± 0.15.

(1) DO

Work in `week8/` inside your existing exercise repository. Every run this week fits on a laptop CPU; the whole week takes about 15 minutes of computing. Send the prompts, then read UNDERSTAND while your agent works.

```
Work in week8/ using uv, with CPU as the default. Use JAX for models and NumPy for
checks. Save figures in evidence/ and raw runs with settings in artifacts/. Keep
downloads and raw runs out of git. Commit after each task; ask about uncertainties
instead of guessing.
```

```
Download the Model2Vec embedding table minishlab/potion-base-8M from Hugging Face at
revision bf8b056 (model.safetensors and tokenizer.json) and load it without PyTorch.
Make a control copy of the table with its rows shuffled by seed 1730. A word's vector
is the row of its token.
```

(2) UNDERSTAND

Embeddings. A network cannot multiply an integer token by a weight in a meaningful way: token 7 is not "larger" than token 3. Instead, each token v gets its own vector, row v of an embedding table $E \in \mathbb{R}^{V \times d}$. Write the one-hot vector of v as e_v, with a one in place v and zeros elsewhere. Then looking up row v is the matrix product

$$E[v] = e_v^\top E. \tag{1}$$

In Figure 3, the single one in $e_v^\top$ picks row 3 of E, and every other row is multiplied by zero. The lookup is therefore a linear layer whose input is a one-hot vector. Parts 3 to 6 learn E by training. This Part looks inside a table that someone else has already trained.

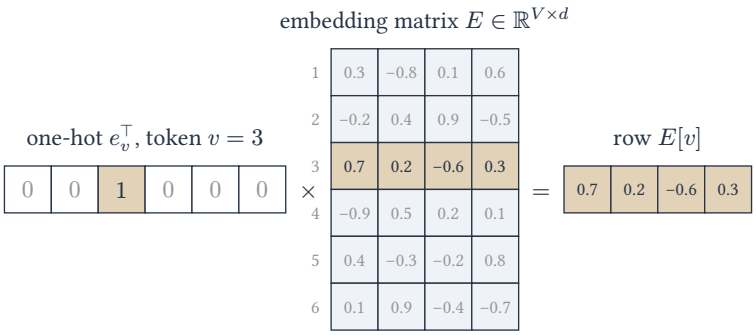


Figure 3: An embedding lookup written as a matrix product. The one-hot row vector of token 3 times the embedding matrix returns row 3.

The pretrained table. `potion-base-8M` from Model2Vec [1] is one such matrix, with $V = 29,528$ rows and $d = 256$ columns, a 30 MB file. Its tokens are word pieces, and common words such as "cat" and "january" are a single token each, so their vector is a single row. The table was distilled from a sentence-embedding network trained on far more text than TinyStories: words used in similar sentences were pushed to similar rows. The nearest rows to "cat", by the angle between vectors, are "cats", "kitty" and "kitten".

Relations as directions. A pair of words such as (france, paris) has an offset, the vector $E[b] - E[a]$ from the first word to the second, with each row first scaled to length one. If the table stores the relation "country to capital" as one direction, the offsets of different pairs point the same way. We test each pair against the others: the cosine between its offset and the average offset of the other eight pairs. It is the cosine of the angle between the two vectors: 1 when the offset points along the others, at 0°, and near 0 when it is unrelated to them, at 90°. Leaving the pair out of the average keeps it from agreeing with itself. Two random directions in 256 dimensions have a cosine of about $\pm\frac{1}{16}$, so a mean near 0.5 is far from chance. Mikolov and coauthors [2] found such parallel offsets in word vectors trained on news text.

The shuffled control. Shuffling the rows keeps every vector and every distance between vectors, but gives each word another word's vector. Any pattern that survives the shuffle comes from the vectors alone, not from what the words mean. A pattern counts as meaning only when the control loses it.

KEY CONCEPTS
- **Embedding table:** one vector per token; a lookup is a one-hot vector times the table.
- **Offset and cosine:** a relation is a direction when each pair's offset points along the average offset of the other pairs.
- **Shuffled control:** the same vectors on the wrong words, so meaning is lost.

3 VERIFY

(1) **A relation between words is one direction in the table.**

```
Take nine word pairs for each of three relations:
- country to capital: iran tehran, turkey ankara, russia moscow, china beijing,
france paris, sweden stockholm, japan tokyo, peru lima, kenya nairobi
- male to female: king queen, man woman, he she, brother sister, father mother, son
daughter, uncle aunt, boy girl, husband wife
- adjective to comparative: big bigger, small smaller, fast faster, dark darker,
cold colder, old older, strong stronger, long longer, tall taller
Scale each row to unit length. For each pair, compute the cosine between its offset
and the average offset of the other eight pairs. For each relation, draw each pair
as a unit arrow whose angle from the horizontal axis has this cosine. Put the
table's arrows above the axis and the control's below. Report each relation's mean
cosine.
```

The table's means must be at least 0.3; the reference gives 0.47, 0.55 and 0.43. The control's must lie within ± 0.15; the reference gives 0.03, -0.01 and 0.01, and no mean over 20 shuffles goes beyond ± 0.09. Almost every blue arrow of the table leans toward the average of the others, while the grey arrows of the control stand near 90°; the weakest pairs are husband⟶wife and old⟶older. Nobody told the model what a capital or a comparative is. This confirms the Goal: the directions of a trained table carry what the words mean, and shuffled rows lose it.

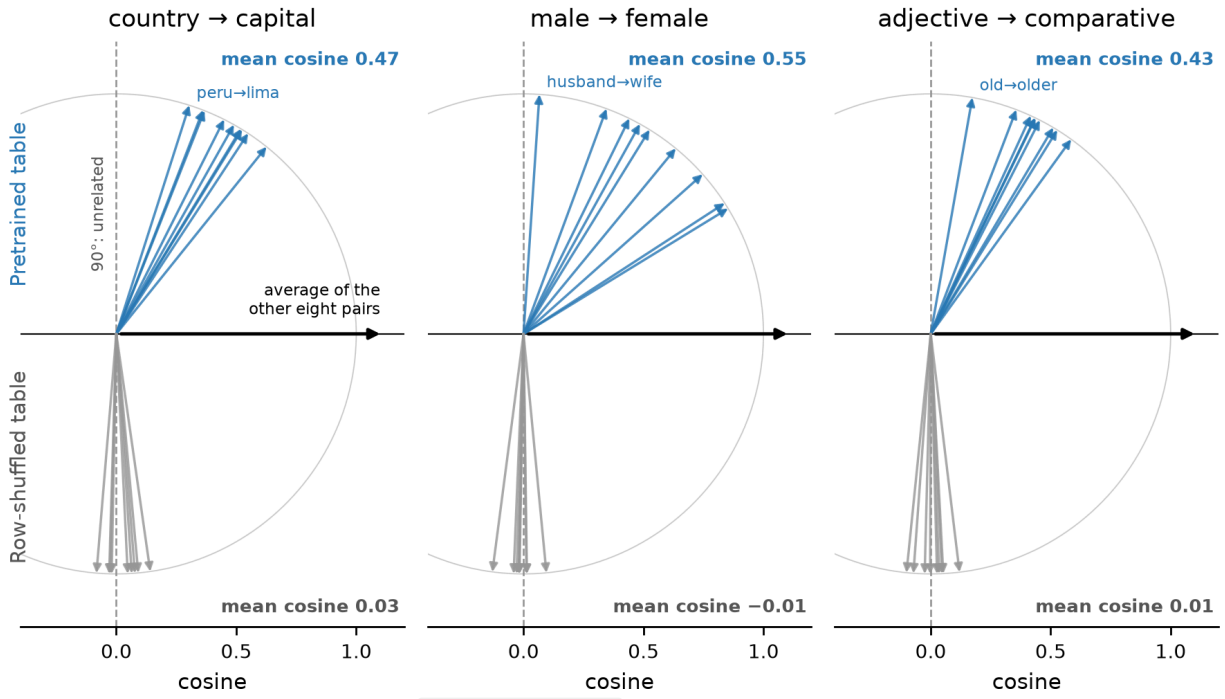


Figure 4: Nine word pairs per relation from `potion-base-8M`. Each arrow is one pair's offset, drawn at its angle to the average offset of the other eight pairs, which points along the black horizontal arrow. The tip's horizontal position is the cosine. Upper half, blue: the table. Lower half, grey: the control with shuffled rows. The dashed line at 90° is an offset unrelated to the others.

Part 2: Count models: Markov chains and n-grams

Goal: model text as a Markov chain on the last few tokens, estimated from counts. Show that these models are exactly normalized and that their samples follow their probabilities. Rank the uniform, unigram, bigram and trigram models by held-out bits per character, about 3.75, 3.14, 1.89 and 1.35, and by perplexity, from 512 down to about 9.5. Show the curse of dimensionality: the 4-gram is about the best, and the 6-gram falls back near the bigram.

(1) DO

Download the corpus yourself:

```
mkdir -p week8/data
curl -L -o week8/data/TinyStoriesV2-GPT4-valid.txt \
  https://huggingface.co/datasets/roneneldan/TinyStories/resolve/f54c09fd23315a6f9c86f
9dc80f725de7d8f9c64/TinyStoriesV2-GPT4-valid.txt
```

```
Split TinyStories in week8/data/ into stories at `<|endoftext|>`. Strip surrounding
whitespace and drop empty stories. Keep the first 90% of stories, in file order, for
training and the rest as held-out validation.
```

```
Download the byte-pair encoding tok512.model from the Hugging Face repository
karpathy/tinyllamas at revision 0bd21da, folder stories260K, and load it with
sentencepiece. Encode both splits, with its end-of-sequence token between stories.
```

```
Build n-gram next-token models, Markov chains on the last n-1 tokens, for n = 1 to 6
from counts of the BPE training tokens. Add the uniform model. Use add-k smoothing,
choosing k from 0.0001, 0.001, 0.01, 0.1 and 1 on the last 10% of training tokens.
Report held-out cross-entropy in nats per token, perplexity and bits per character.
```

(2) UNDERSTAND

The TinyStories corpus and its split. TinyStories [3] holds short stories written by GPT-4 in the vocabulary of a three-year-old. We use one fixed revision of one file, 22.5 MB of plain text with `<|endoftext|>` between stories.

Models are fitted to *training* data and judged on data they never saw. The first 90% of whole stories train every model this week; the remaining 10% are held-out validation: 24,867 and 2,763 stories.

The chain rule and exact normalization. Any probability of a sequence factors into next-token probabilities, each conditioned on everything before it:

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}). \tag{2}$$

This identity, the *chain rule*, needs no assumption. A model that predicts each token from the ones before it, and generates text one token at a time, is *autoregressive*. Each factor sums to one over the V tokens. Summing the product over x_T, then over x_{T-1}, and so on, gives one: the model is normalized for every length, with no Week 6 normalizer to estimate.

Markov chains. A table indexed by every possible history would need a row for each one. A *Markov model* of order m assumes that the next token depends only on the last m tokens:

$$p(x_t \mid x_1, \ldots, x_{t-1}) = p(x_t \mid x_{t-m}, \ldots, x_{t-1}). \tag{3}$$

The assumption is false for real text, since a story's dog keeps its name for the whole story, but it leaves a finite table. Markov chains of order one are the simple case. Such a chain on the V tokens is described by a *transition matrix* P with $P_{ab} = p(b \mid a)$: every entry is non-negative and every row sums to one. A chain of order m is again a simple chain, whose states are the V^m blocks of the last m tokens.

n-gram models. An n-gram model is the Markov model of order $n-1$, so it looks at blocks of n tokens. The *unigram* model ($n = 1$) ignores the context, the bigram model ($n = 2$) is the simple Markov chain, and the *trigram* model ($n = 3$) conditions on two tokens. The uniform model, outside the family, gives every token $\frac{1}{V}$. The end-of-sequence token between stories is the context of each story's first token, so the product starts at $p(x_1 \mid \text{end of sequence})$.

The likelihood and the count estimate. The *likelihood* of a model is the probability it gives the training tokens. Write $n(ab)$ for the number of times the pair ab occurs in them. For the bigram, the log-likelihood groups by pair:

$$\ln p(\text{training tokens}) = \sum_{t} \ln P_{x_{t-1}x_t} = \sum_{a,b} n(ab) \ln P_{ab}. \tag{4}$$

Maximizing it while each row of P sums to one gives

$$P_{ab} = \frac{n(ab)}{n(a)}, \tag{5}$$

the counts of each row divided by their sum. This *maximum-likelihood* estimate holds for every n, with blocks of $n-1$ tokens as rows.

Add-k smoothing. An n-gram that never occurs in training gets probability zero, and one such n-gram in held-out text makes the probability of the whole held-out text zero. With add-k smoothing, every count gets k extra:

$$p(b \mid a) = \frac{n(ab) + k}{n(a) + kV}. \tag{6}$$

It gives up a little training likelihood to keep every probability positive. A large k pulls each row toward uniform; a small k trusts counts seen only once or twice. We choose k on the last 10% of the training tokens, never on validation.

Cross-entropy loss. A model is judged by the average negative log probability it gives the held-out tokens, the cross-entropy loss:

$$L = -\frac{1}{N} \sum_{t=1}^{N} \ln p(x_t \mid x_{<t}), \tag{7}$$

the negative log-likelihood of the held-out tokens per token. With the natural logarithm its unit is the nats per token; dividing by $\ln 2$ converts to bits. The total in bits divided by the number of held-out characters gives bits per character. Tokens cover different numbers of characters, so this unit compares models across tokenizers.

The KL divergence. Write q for the true distribution of the next token after a given context, and p for the model's. Averaged over the contexts of a large held-out set, L estimates the cross-entropy of p relative to q:

$$H(q,p) = -\sum_x q(x)\ln p(x) = \underbrace{-\sum_x q(x)\ln q(x)}_{H(q)} + \underbrace{\sum_x q(x)\ln\frac{q(x)}{p(x)}}_{D_{\mathrm{KL}}(q\,\|\,p)}. \tag{8}$$

The entropy $H(q)$ belongs to the text; no model changes it. The *Kullback–Leibler divergence* $D_{\mathrm{KL}}(q \,\|\, p)$ is never negative and is zero only when $p = q$. A lower loss therefore means a model closer to the text, and the entropy of the text is a floor no loss goes below.

Perplexity. The perplexity is e^L, with L in nats per token. A model that spreads its probability evenly over K tokens has $L = \ln K$ and perplexity K. Perplexity is therefore the number of equally likely tokens the model chooses between on average; the uniform model has $L = \ln V$ and perplexity V.

Ancestral sampling. To generate, draw x_1, then x_2 from $p(\cdot \mid x_1)$, and so on, each draw using the tokens already drawn. This is ancestral sampling, drawn as a loop in Figure 5; for the bigram it is a walk of the Markov chain. Every draw uses exactly one factor of the chain rule, so the samples have exactly the model's distribution. Part 5 runs the same loop with a transformer in place of the counts.

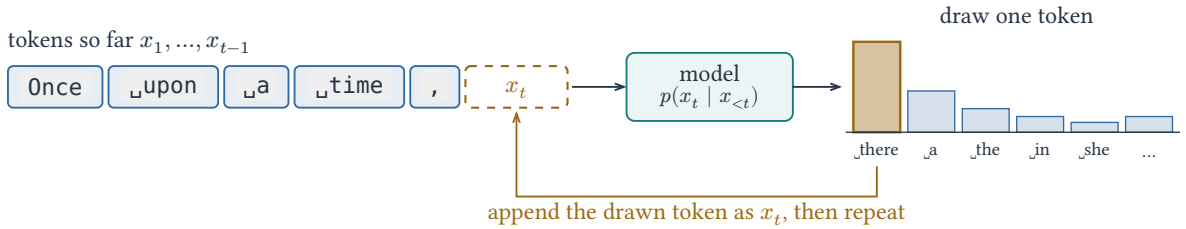


Figure 5: Ancestral sampling. The model turns the tokens so far into a next-token distribution; one token is drawn from it and becomes the last token of the context for the next draw.

The chi-squared test. To compare drawn counts O_i with expected counts E_i, the chi-squared test uses the statistic $\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$, which is small when they agree. Its *p-value* is the probability that a correct sampler gives a statistic at least this large; below 0.001 it signals a wrong sampler. The test needs every expected count to be at least about 5, so rare next tokens are grouped into one bin. Tests of several contexts combine by adding their statistics and their degrees of freedom.

The curse of dimensionality. An n-gram table has one row for each of the V^{n-1} contexts: 512 for the bigram, 2.6×10^5 for the trigram and 3.5×10^{13} for the 6-gram. The training split holds only 8.3×10^6 tokens, so almost every row of a long-context table is empty. This growth of the space faster than any data set can fill it is the curse of dimensionality. Smoothing gives a context never seen in training the uniform guess $\frac{k}{kV} = \frac{1}{V}$. A longer context lowers the KL divergence of the best model of its order, but raises the error of estimating that model from counts. Past some n the second effect wins and the held-out loss rises. Counts also share nothing between similar contexts: "the dog ran" and "the cat ran" are unrelated rows.

KEY CONCEPTS

- **Chain rule:** a sequence's probability is a product of next-token probabilities, so the model is normalized.
- **Markov chain and n-gram:** the n-gram model keeps the last $n - 1$ tokens; the bigram is the simple Markov chain, a transition matrix.
- **Likelihood and smoothing:** row-normalized counts maximize the training likelihood; add-k smoothing keeps every probability positive.
- **Cross-entropy, KL divergence, perplexity:** the held-out negative log-likelihood per token is the text's entropy plus the model's KL divergence; its exponential counts the equally likely next tokens.
- **Ancestral sampling:** draw each token from the model given those already drawn.
- **Curse of dimensionality:** V^{n-1} contexts outgrow any training set, so long-context counts are mostly empty.

3 VERIFY

(1) **The bigram model is exactly normalized.**

```
Sum the bigram model's probability over every token sequence of length 1, 2 and 3,
by explicit enumeration. Compare each sum with one.
```

Each sum must equal one within 10^{-12}; the reference error is at most 6.7×10^{-16}. Scores that are never divided by their row sum, such as $\exp(n(ab)/\max n)$, sum to 512, 2.6×10^5 and 1.3×10^8 instead.

(2) **Longer context lowers the held-out loss up to the trigram.** The split must hold 24, 867 training and 2, 763 validation stories, and the byte-pair encoding about 2.05 characters per token, since every value below depends on both. Decoding returns every validation story except 3, in which the tokenizer merged a double space into one. The run of the n-gram prompt reports these values. Losses should lie within 0.03 of each, and perplexities within 3%.

Model	Nats per token	Perplexity	Bits per character
uniform	6.238	512	3.747
unigram ($n = 1$)	5.230	186.8	3.141
bigram ($n = 2$)	3.152	23.4	1.893
trigram ($n = 3$)	2.254	9.53	1.354

The uniform row is exact. Each extra token of context divides the perplexity: the trigram chooses among about ten tokens where the uniform model chooses among 512.

(3) **Samples follow the bigram's probabilities.**

> `Generate 250000 tokens from the bigram model by ancestral sampling. Compare the next-token counts after the 20 most frequent tokens with the model by one chi-squared test pooled over these contexts, and report the p-value.`

The p-value must exceed 0.001; the reference gives 0.25. A sampler that conditions on the token two places back gives $p = 0$.

(4) **The curse of dimensionality stops the n-gram ladder.**

> `Measure the curse of dimensionality of the n-gram models. For n = 1 to 6, count the fraction of held-out n-grams that never occur in the training tokens. Plot it and the held-out bits per character against n.`

The loss must be lowest at $n = 3$ or 4; the reference gives 1.354 and 1.315 bits per character. It must then rise, to 1.56 at $n = 5$ and 1.93 at $n = 6$, within 0.03 of each. The unseen fractions should lie within two percentage points of 1.7%, 8.6%, 21.1% and 35.7% for $n = 3$ to 6. At $n = 6$, more than a third of the held-out tokens are predicted by smoothing alone, and the 6-gram, with perplexity 25.0, predicts no better than the bigram. This confirms the Goal's last clause.

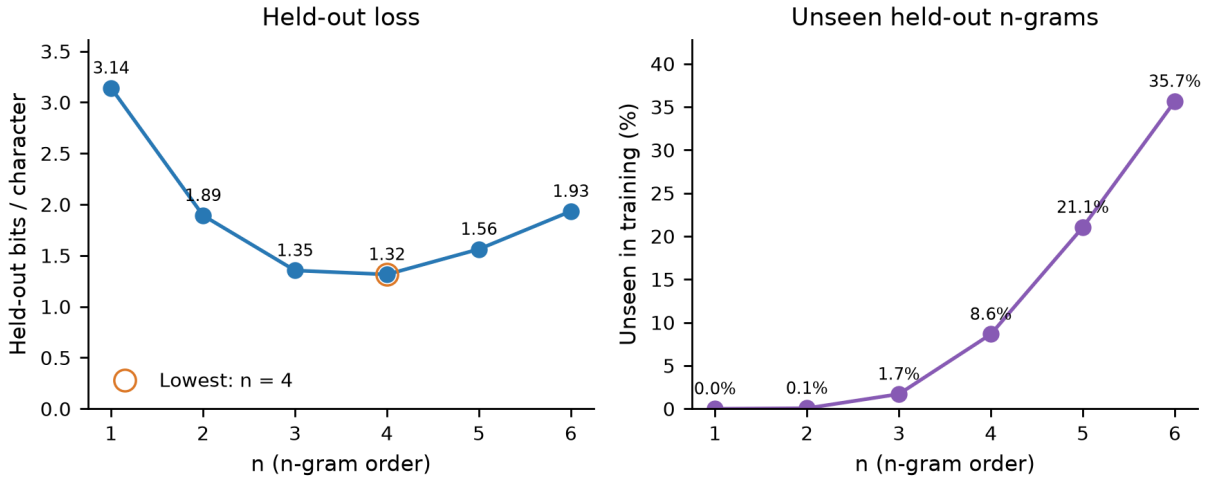


Figure 6: Reference n-gram models for $n = 1$ to 6. Left: held-out bits per character, lowest at $n = 4$. Right: the percentage of held-out n-grams that never occur in the training tokens.

Part 3: The MLP: a memory in the weights

Goal: build memories that store pairs of vectors, a key and a value, and recall a value by the similarity of its key to a question. Show that a store in one matrix holds about d memories in d dimensions, and that a two-layer network with ReLU^p holds about d^p. Train such a network as a language model and show that it stores the bigram table, matching the bigram's held-out loss within 0.01 nats per token.

1 DO

```
Build key-value memory stores in NumPy, with M random unit keys and values in d
dimensions. Each store recalls a query x as a weighted sum of values. Use the outer-
product store, the least-squares linear store, and MLP stores weighted by
ReLU(similarity)^p for p = 1, 2 and 3.
```

```
For d = 16, 32 and 64 and M from 16 to 65536 in powers of two, query every store with
stored keys. Measure the interference-to-signal ratio: the norm of the other memories'
contribution over the norm of the stored value's own term. Average over 3 independent
draws, and report each store's capacity.
```

The stores hold random facts. Now train a network that must store real ones.

```
Train an MLP next-token model on the previous BPE token: a 64-dimensional embedding,
one hidden layer of 512 tanh units. Use 4000 AdamW steps of 16384 examples, learning
rate 0.006 with warmup and cosine decay. Report held-out cross-entropy in nats per
token.
```

2 UNDERSTAND

Facts a language model must remember. Part 2's bigram kept its facts in a table of counts. A better model needs two kinds of facts. Some hold in every story and can be learned once, during training: "once upon a" is followed by "time". Others hold only in the story being read: the dog in this story is called Max. Parts 3 and 4 build a memory for each kind, from nothing but vectors, sums and products. Part 5 then joins the two memories. The result is the *transformer*, the network behind today's language models.

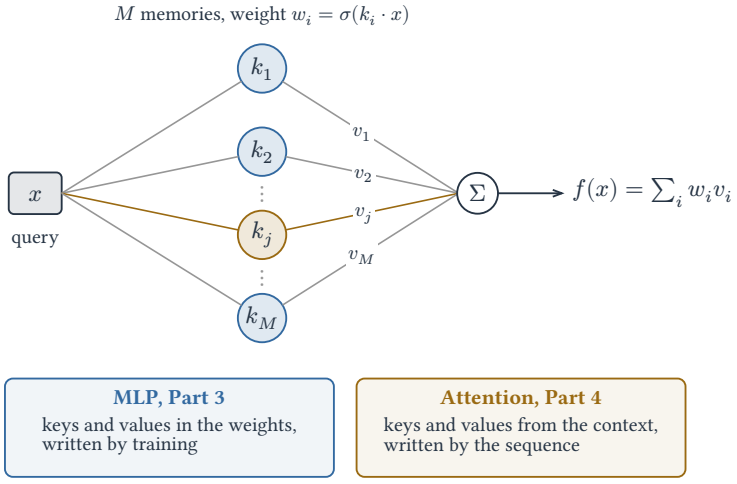


Figure 7: One key–value memory store. The query x is compared with every key; each memory's value enters the answer with weight $w_i = \sigma(k_i \cdot x)$. The highlighted memory has the key closest to the query. Part 3 fills the store during training; Part 4 fills it from the text being read.

Finding a memory by similarity. A Python dictionary stores pairs such as `"cat": "meow"`. It finds a pair only when the question equals the key exactly. A network works with vectors, and two vectors are rarely exactly equal. It therefore finds memories by similarity. For two vectors of length one, we measure similarity by their dot product, $k \cdot x = \sum_a k_a x_a$. It is 1 for equal vectors, 0 for perpendicular vectors and -1 for opposite vectors.

A key–value memory store. A key-value memory store holds M memories. Memory i is a pair of vectors. Its key k_i, of length one, is compared with questions. Its value v_i is the content returned. A question is a vector x, called the query. The store answers with a weighted sum of all its values (Figure 7):

$$f(x) = \sum_{i=1}^{M} w_i v_i, \quad w_i = \sigma(k_i \cdot x). \tag{9}$$

The function σ turns each similarity into a weight. A good store gives a large weight to the memory whose key matches the query and almost no weight to the others. Asking with a stored key k_j then returns its value v_j. The stores of Parts 3 and 4 differ only in the function σ.

The outer-product store. The simplest choice uses the similarity itself as the weight, $w_i = k_i \cdot x$. The whole store then fits in one matrix. The outer-product store adds one matrix $v_i k_i^\top$ for each memory:

$$W = \sum_{i=1}^{M} v_i k_i^\top, \quad Wx = \sum_{i=1}^{M} v_i (k_i^\top x) = \sum_{i=1}^{M} (k_i \cdot x) v_i. \tag{10}$$

The matrix $v_i k_i^\top$, the *outer product* of value and key, has the entry $v_{ia} k_{ib}$ in row a and column b. Multiplying a vector by a matrix is the basic step of every neural network, called a *linear layer*. So a linear layer can hold memories. Training writes them in this very form. For a linear layer $y = Wx$, one gradient step adds to W the outer product of $-\eta \frac{\partial L}{\partial y}$ and the input x, where L is the loss and η the learning rate. Week 6's Hopfield network stored its patterns by the same rule.

Recall with perpendicular keys. In three dimensions, three keys can lie along the three axes. Each key then has similarity 0 with the other two, and $Wk_j = v_j$ exactly. A fourth key cannot be perpendicular to all three. Asking with it returns its value plus parts of the other values. In d dimensions, at most d keys are perpendicular to one another.

Signal and interference. With more keys than dimensions, asking with k_j returns

$$f(k_j) = \underbrace{v_j}_{\text{signal}} + \underbrace{\sum_{i \neq j} (k_i \cdot k_j) v_i}_{\text{interference}}. \tag{11}$$

The first term is the wanted value. The second term is the leak from the other memories. The interference-to-signal ratio R divides the length of the leak by the length of the wanted value. Recall is reliable while R is well below one.

The growth of the interference. Two keys drawn at random in d dimensions are nearly perpendicular. Their similarity is a small random number, positive or negative, of typical size $\frac{1}{\sqrt{d}}$: about 0.125 at $d = 64$. Each other memory therefore leaks a piece of length about $\frac{1}{\sqrt{d}}$ in a random direction. Pieces in random directions partly cancel, and their squared lengths add. So $M - 1$ pieces have a total length of about $\sqrt{\frac{M}{d}}$:

$$R \approx \sqrt{\frac{M}{d}}. \tag{12}$$

The store's capacity is the number of memories at which $R = 1$, here about d. Doubling the dimension doubles the number of memories. The Hopfield network of Week 6 likewise stored a number of patterns proportional to its number of neurons, 0.138 per neuron.

The least-squares linear store. The outer-product store writes each memory without looking at the others. The least-squares linear store instead picks the matrix whose answers to the stored keys come closest to their values:

$$W = \arg\min_{W} \sum_{i=1}^{M} \|Wk_i - v_i\|^2. \tag{13}$$

With $M \leq d$ keys it answers every stored key exactly, $R = 0$, as perpendicular keys do. Beyond d keys, $R \approx \sqrt{\frac{M}{d} - 1}$, so the capacity is about $2d$. No matrix does much better. A matrix answers in proportion to its input, so it cannot ignore a memory that is only a little similar to the query.

Powers of ReLU. A function that is not proportional can. ReLU, $\sigma(s) = \max(0, s)$, sets the negative similarities to zero. Half of the leaking memories fall silent, and the capacity doubles to about $2d$. Raising ReLU to a power p also shrinks the small positive similarities, while the matching memory keeps the weight $1^p = 1$. At $d = 64$, a typical similarity of 0.125 becomes 0.016 at $p = 2$ and 0.002 at $p = 3$. Averaging the squared leak over random similarities gives

$$R \approx \sqrt{\frac{(2p-1)!!\, M}{2d^p}}, \quad \text{capacity} \approx \frac{2d^p}{(2p-1)!!}, \tag{14}$$

where $(2p-1)!! = 1 \cdot 3 \cdots (2p-1)$ equals 1, 3 and 15 for $p = 1, 2, 3$. The capacity grows like d^p: at $d = 64$, about 128, 2,731 and 34,953 memories. These estimates treat each similarity as a Gaussian random number of variance $\frac{1}{d}$.

The MLP. The store with a nonlinear σ is a small neural network:

$$f(x) = V^\top \sigma(Kx). \tag{15}$$

The rows of the matrix K are the keys, and the rows of V are the values. The network multiplies by one matrix, applies σ to each entry, and multiplies by a second matrix. This network is an MLP, a *multilayer perceptron*, with one hidden layer. Each entry of $\sigma(Kx)$, a *hidden unit*, belongs to one memory. The transformer of Part 5 contains such an MLP, with $4d$ hidden units and a smooth version of ReLU. Geva and coauthors [4] looked inside trained transformers and found units that behave like memories: a unit responds to a recognizable input pattern, and its output raises the probability of related next tokens.

An MLP that predicts the next token. Bengio and coauthors [5] used an MLP as a language model. It looks up the embedding $h_0 = E[x_t] \in \mathbb{R}^d$ of the previous token (Part 1) and computes

$$h_1 = \tanh(W_1 h_0 + b_1), \quad z = W_2 h_1 + b_2, \quad p(v \mid x_t) = \frac{\exp(z_v)}{\sum_{u=1}^{V} \exp(z_u)}. \tag{16}$$

The vectors b_1 and b_2 are *biases*, learned shifts. The function tanh squashes each entry into $(-1, 1)$. The output z holds one score per token of the vocabulary, called a *logit*. The last formula, the *softmax*, makes the scores positive and divides them by their sum, which gives a probability distribution. Training adjusts every weight to lower the cross-entropy of Part 2 on the training tokens. Read as a store, the previous token is the query, the rows of W_1 are the keys, and the columns of W_2 are the values: the scores that each hidden unit adds.

The one-token MLP and the bigram. The model sees exactly what the bigram sees. It must write the bigram table, one next-token distribution for each token, into its weights. No model that sees one token can predict better than the bigram, so a well-trained MLP reaches the bigram's held-out loss, neither better nor worse. This is a check on training, not a feature of the network.

Training with AdamW. An optimizer turns gradients into weight updates. Week 7's Adam scaled each update by running averages of the gradient and its square. AdamW adds *weight decay*, a small pull of every weight toward zero. Each *step* updates the weights from one batch of examples. Large early updates can throw a freshly started network off course, so warmup raises the learning rate from near zero over the first steps. Cosine decay then lowers it along half a cosine period, so the last updates are small. The learning rate in a prompt is the peak between the two.

Facts stated only in the context. After training, the weights stay fixed. A fact that appears only in the current story, such as the dog's name, was never written into them. A memory for such facts must be written by the text itself. Part 4 builds it.

KEY CONCEPTS

- **Key–value memory store:** a query is compared with every key; the answer sums the values, weighted by similarity.
- **Interference and capacity:** other memories leak into an answer; the capacity is where the leak equals the wanted value.
- **Outer-product and least-squares stores:** one matrix holds about d or $2d$ memories; a gradient step writes an outer product.
- **MLP:** matrix, ReLU, matrix; each hidden unit is a memory, and ReLU^p holds about d^p.
- **MLP language model:** previous token's embedding, tanh layer, softmax; it stores the bigram table.
- **Facts in the context:** fixed weights cannot hold them; they need a memory written by the text.

3 VERIFY

(1) **The interference-to-signal ratio grows like $\sqrt{M}$.**

```
Plot the interference-to-signal ratio against the number of memories for every
store at d = 64, on log-log axes, with the predicted lines.
```

Every curve must rise with slope $\frac{1}{2}$ once R passes about 0.1. The outer-product store must reach $R = 1$ between $M = 50$ and 85; the reference crosses at 65.5. The least-squares store must give R below 10^{-10} up to $M = 64$ and cross 1 between 100 and 170; the reference gives 5×10^{-14} and 129. ReLU^2 and ReLU^3 cross near 2,889 and 38,800, a little above the predicted 2,731 and 34,953: the Gaussian estimate overstates how often two unit vectors are similar.

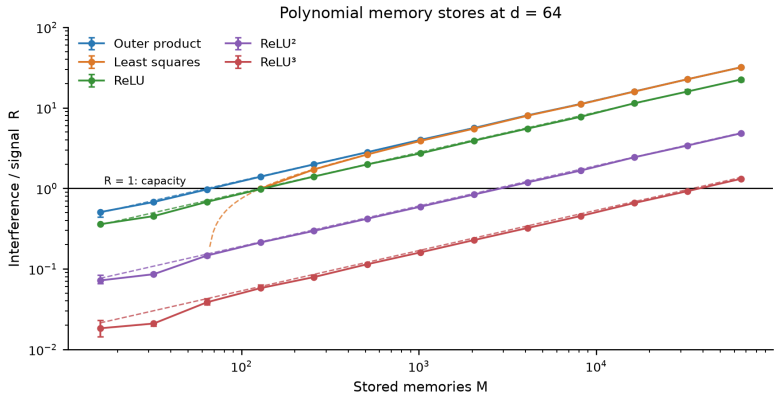


Figure 8: Interference-to-signal ratio of the reference stores at $d = 64$, median of three draws. Dashed lines are the predictions of UNDERSTAND; the horizontal line marks the capacity.

(2) **Capacity grows like d^p.**

> Plot each store's capacity against d on log-log axes, and fit its slope.

The outer-product, least-squares and ReLU slopes must lie within 0.1 of 1; the reference gives 1.02, 0.99 and 0.99. The ReLU^2 slope must lie within 0.15 of 2, and the ReLU^3 slope must exceed 2.5; the reference gives 1.93 and 2.78. A similarity of two unit vectors never exceeds one, so at $d = 16$ large similarities are rarer than the Gaussian estimate assumes: ReLU^3 stores 817 memories instead of the predicted 546, which flattens its slope. Keys that share a common direction break the law: with a mean similarity of 0.3, the outer-product store holds about 11 memories at every d.

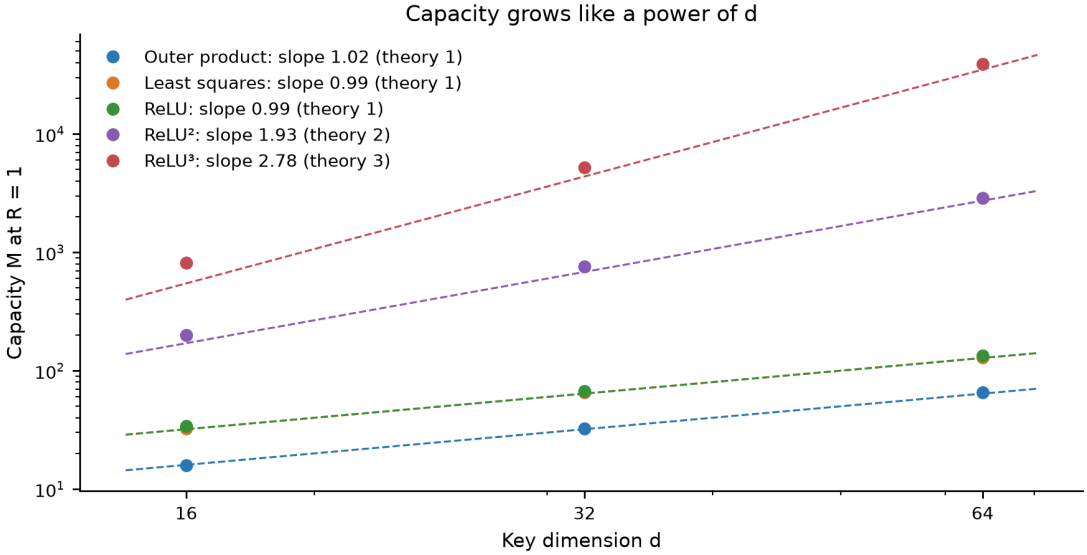


Figure 9: Capacity of the reference stores against d, median of three draws, with fitted lines. Each power p of ReLU multiplies the slope.

(3) **The MLP stores the bigram table.** The held-out loss must lie within 0.01 nats per token of the bigram loss of Part 2, 3.152; the reference gives +0.005. With 512 examples per step the same model stays 0.03 nats above: the gradient of a small batch is too noisy to settle on the optimum within 4000 steps.

Extension: the hidden units of the one-token MLP read as memories. Each of the 512 tanh units of your MLP language model has a key, the previous tokens that drive it, and a value, the next tokens its column of W_2 raises. Tanh is odd, so each unit has a positive and a negative side. List the five strongest previous and next tokens of each side, and measure the share of a side's squared activation that its five strongest previous tokens carry. In the reference, the units are not clean memories. The median share is 0.12, below the 0.135 of the untrained network, and even the most concentrated sides mix word pieces. The model stores 512 bigram rows in 512 shared units, and spreads each row over many units. Geva and coauthors [4] found readable memories in large trained transformers, and Cammarata [6] argues, from a preliminary and disputed audit, that most units of a large model have one main job. The capacity argument of this Part does not depend on whether a unit is readable.

Hidden units of the one-token MLP read as key → value memories
median top-5 share 0.118 trained, 0.135 untrained

Unit, side	Top-5 share	Key: previous tokens that excite it		Value: next tokens it promotes
337 +	0.29	u ·an el ·ro ·lo	→	ted ly ry ri her
337 −	0.05	ime ·that ▯ : ·fun	→	·y ·wh ·cl ·tre ·cat
263 +	0.28	·sa 3 ·l q l	→	om 's m l ound
263 −	0.07	V ·sad : ·played ·tree	→	on ·ha ·help ar ·l
494 +	0.28	E Once H or 3	→	·have ore ld ·at ill
494 −	0.07	ith ·It ig z ·dog	→	!" es ·sp z ·do
466 +	0.27	·sh X = ·d in	→	et ·we ll ic out
466 −	0.07	" ·too ily ·Tim ith	→	·help ·with ry ly ·play

Figure 10: The four reference units whose positive side is most concentrated. A middle dot marks a space.

Part 4: Attention: a memory built from the text

Goal: build a memory whose key–value pairs come from the text being read. Show that exponential weights silence the other memories: a picture stored among $4,096$ random memories in $d = 64$ comes back clean from the softmax store at sharpness $\beta = 8$, the sharpness set by the $\frac{1}{\sqrt{d}}$ scale, while the stores of Part 3 lose it. Show that one attention layer, reading the whole sequence, recalls the queried value at every distance.

① DO

```
Store a 32 by 32 picture of the character ▯ among M memories in d = 64, with pixels +1
for ink and -1 for background. Give the other memories random ±1 pictures and all
memories random unit keys. Recall the picture with its key from the outer-product,
ReLU, ReLU^2 and softmax stores at sharpness beta = 8, for M = 64, 256, 1024 and 4096.
```

```
Build a recall task. A sequence holds four memories, each one token joining one of 8
keys with one of 8 values, then a query token naming a stored key. The target is that
key's value. Draw new pairs for every sequence.
```

```
Implement single-head self-attention in JAX, with query, key, value and output
matrices and the 1/sqrt(d) scale. For the recall task, embed tokens in width 64
without positions, add the query's attention output to its embedding, layer-normalize,
and read 8 value logits. Train 4000 AdamW steps of 256 sequences at learning rate
0.003.
```

2 UNDERSTAND

A memory written by the text. The MLP's memories are fixed once training ends. Attention builds a fresh memory store from the text being read. Every token read so far adds one memory, a key and a value computed from that token's vector. The current position asks with its query. A story of 100 tokens gives 100 memories, and a longer story gives more, with no extra weights. Because the store can grow large, its weighting must silence the many unrelated memories.

The recall task. The recall task tests such a store with artificial sequences. There are 8 keys and 8 values. Each of four memories is one token standing for a key–value pair, such as "key 0 has value 3", and the four keys differ. A final query token names one of the stored keys, and the target is its value. The pairs are drawn anew for every sequence, so no weight can store them. The distance D counts how many positions the queried memory lies before the query. Guessing gives accuracy $\frac{1}{8}$.

The exponential store. Part 3 found that a higher power p silences small similarities better. The exponential function contains every power at once:

$$e^{\beta s} = 1 + \beta s + \frac{(\beta s)^2}{2} + \frac{(\beta s)^3}{6} + \cdots = \sum_{p=0}^{\infty} \frac{(\beta s)^p}{p!}. \tag{17}$$

This sum is its *Taylor series*. The *exponential store* uses the weights $w_i = \exp(\beta k_i \cdot x)$. The number β, the sharpness, sets how steeply the weight rises with similarity. At $\beta = 8$ the matching key, with similarity 1, gets the weight $e^8 \approx 2,981$. A typical other key at $d = 64$, with similarity 0.125, gets $e^1 \approx 2.7$, a thousand times less. Averaging over random similarities gives

$$R \approx \sqrt{M}\, e^{\beta^2/d - \beta}, \quad \text{capacity} \approx e^{2\beta(1-\beta/d)}. \tag{18}$$

At $d = 64$ this predicts about 48 memories for $\beta = 2$, $1,808$ for $\beta = 4$ and 1.2 million for $\beta = 8$, against $2,731$ for ReLU^2. The capacity grows exponentially with the sharpness.

Softmax normalization. The exponential grows fast. At $\beta = 8$ the weights reach thousands, and larger scores overflow the computer's numbers. Dividing every weight by the sum of all weights keeps them between zero and one:

$$f(x) = \sum_{i=1}^{M} \frac{\exp(\beta k_i \cdot x)}{\sum_{l=1}^{M} \exp(\beta k_l \cdot x)} v_i = V^\top \operatorname{softmax}(\beta K x). \tag{19}$$

This is the softmax of Part 3, and the store is the softmax store. The division scales all weights of one query by the same number, so their ratios, and the ratio R, stay the same. The answer is now an average of the values, weighted by similarity.

Recalling a picture. A value can be any vector, so it can be a picture. A 32×32 picture with pixels $+1$ for ink and -1 for background is a value with $1,024$ entries. Store it as one memory among M, give the others random ± 1 pictures, and ask with its key k_j. Divided by the picture's own weight w_j, the answer is the stored picture plus the leak of the other memories, pixel by pixel. While R is well below one the character stands out; once R passes one the leak hides it. The formulas predict where. At $d = 64$ and $M = 4,096$, the outer-product store gives $R \approx 8$, ReLU about 5.7, ReLU^2 about 1.2, and the softmax store at $\beta = 8$ about 0.06. Wang [7] draws the same comparison with another character.

Queries, keys and values from the tokens. In self-attention [8], the memories come from the sequence itself. Each position t holds a vector $x_t \in \mathbb{R}^d$, such as its token's embedding. The vector size d is the layer's width. Three learned matrices turn each vector into three new ones: a query $q_t = W_Q x_t$, a key $k_t = W_K x_t$ and a value $v_t = W_V x_t$. Training chooses the matrices, so the network learns what each position asks for, what it is found by, and what it returns. Position t weighs every position j by the similarity of its query and the key k_j:

$$a_{tj} = \frac{\exp\left(q_t \cdot k_j / \sqrt{d}\right)}{\sum_i \exp\left(q_t \cdot k_i / \sqrt{d}\right)}, \quad o_t = W_O \sum_j a_{tj} v_j. \tag{20}$$

A fourth matrix W_O maps the averaged value back into the vector space of the positions. These are the query, key, value and output matrices W_Q, W_K, W_V, W_O. On the recall task, the query token can learn to match the memory with the same key. In Figure 11, the query puts 0.86 of its weight on the memory with key 0, wherever that memory sits, and reads that memory's value.

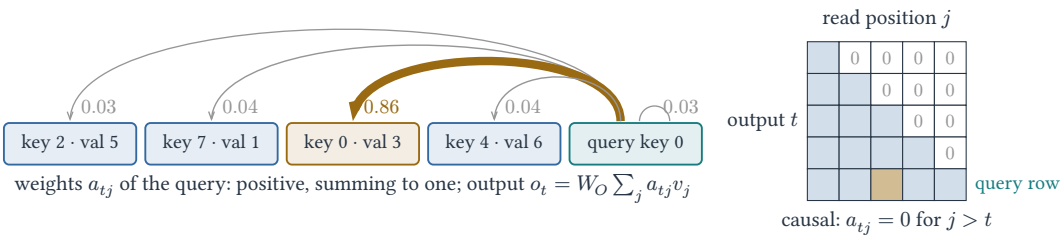


Figure 11: Left: the query position reads every earlier position, with arrow width proportional to its attention weight. Right: the causal mask of Part 5; row t may put weight only on columns $j \le t$.

The $\frac{1}{\sqrt{d}}$ scale as the sharpness. Learned queries and keys are not vectors of length one. If their entries are random numbers of variance one, the dot product $q \cdot k$ has variance d, so its typical size grows like $\sqrt{d}$. The 1/sqrt(d) scale divides the scores by $\sqrt{d}$, which keeps their typical size at one for every width. Vectors of length about $\sqrt{d}$ then behave like the exponential store with $\beta = \sqrt{d}$, which is $\beta = 8$ at $d = 64$. A larger β would store even more, but it would put nearly all weight on one position. The gradient through the other positions would then nearly vanish, and training could not discover which position to read.

The recall readout. The recall model adds the attention output at the query position to the query's own embedding, so attention only has to supply what the query lacks. It then rescales that vector to mean zero and variance one across its entries, which keeps its size steady during training. A final matrix maps it to 8 scores, one per value.

The memory stores of Week 6. Week 6 climbed the same ladder. The classical Hopfield network is the outer-product store. Dense associative memories with a polynomial energy [9] store a power of the dimension, and an exponential energy [10] stores exponentially many patterns. The modern Hopfield update of Week 6 [11] is the softmax store, with the stored patterns as keys and values and the probe as the query.

Attention and token order. Attention sees a set of memories, not their order. Shuffling the earlier positions shuffles their weights in the same way and leaves the sum unchanged. The recall task needs no order, since each memory carries its own key. Part 5 adds positions for text, where order matters.

KEY CONCEPTS

- **Memory written by the text:** each token read adds one memory; the store grows with the story.
- **Exponential store:** every power at once; the capacity grows exponentially with the sharpness β.
- **Softmax store:** the exponential weights divided by their sum; the ratio R is unchanged.
- **Recalling a picture:** the answer is the stored picture plus the leak; it stays readable while R is below one.
- **Query, key, value:** three learned maps of each position's vector; attention is the softmax store over them.
- **The $\frac{1}{\sqrt{d}}$ scale:** sets the sharpness $\beta = \sqrt{d}$, selective enough to store many memories and soft enough to train.

3 VERIFY

(1) **The softmax store recalls the picture among $4,096$ memories.**

```
Draw each recalled picture as dots sized by its pixels, divided by the stored
picture's own weight: one row per M and one column per store. Print the
interference-to-signal ratio of each recall.
```

At $M = 1,024$ the outer-product and ReLU ratios must lie within 15 percent of Part 3's predictions $\sqrt{\frac{M}{d}} = 4$ and $\sqrt{\frac{M}{2d}} = 2.83$; the reference gives a median of 4.05 and 2.84 over 20 draws. The character is readable only while R is below about one. The outer-product store barely shows it at $M = 64$, and ReLU loses it from $M = 256$. ReLU^2 keeps it up to $M = 1,024$, at $R = 0.60$ against the predicted 0.61. At $M = 4,096$, beyond its capacity of $2,731$, its ratio must exceed one; the reference gives 1.2. The softmax store must stay below $R = 0.1$ at every M; the reference gives 0.057 at $M = 4,096$, against the predicted $\sqrt{M}\, e^{\beta^2/d - \beta} = 0.058$. Its picture looks the same in every row, since its capacity of 1.2 million lies far beyond the largest store here.

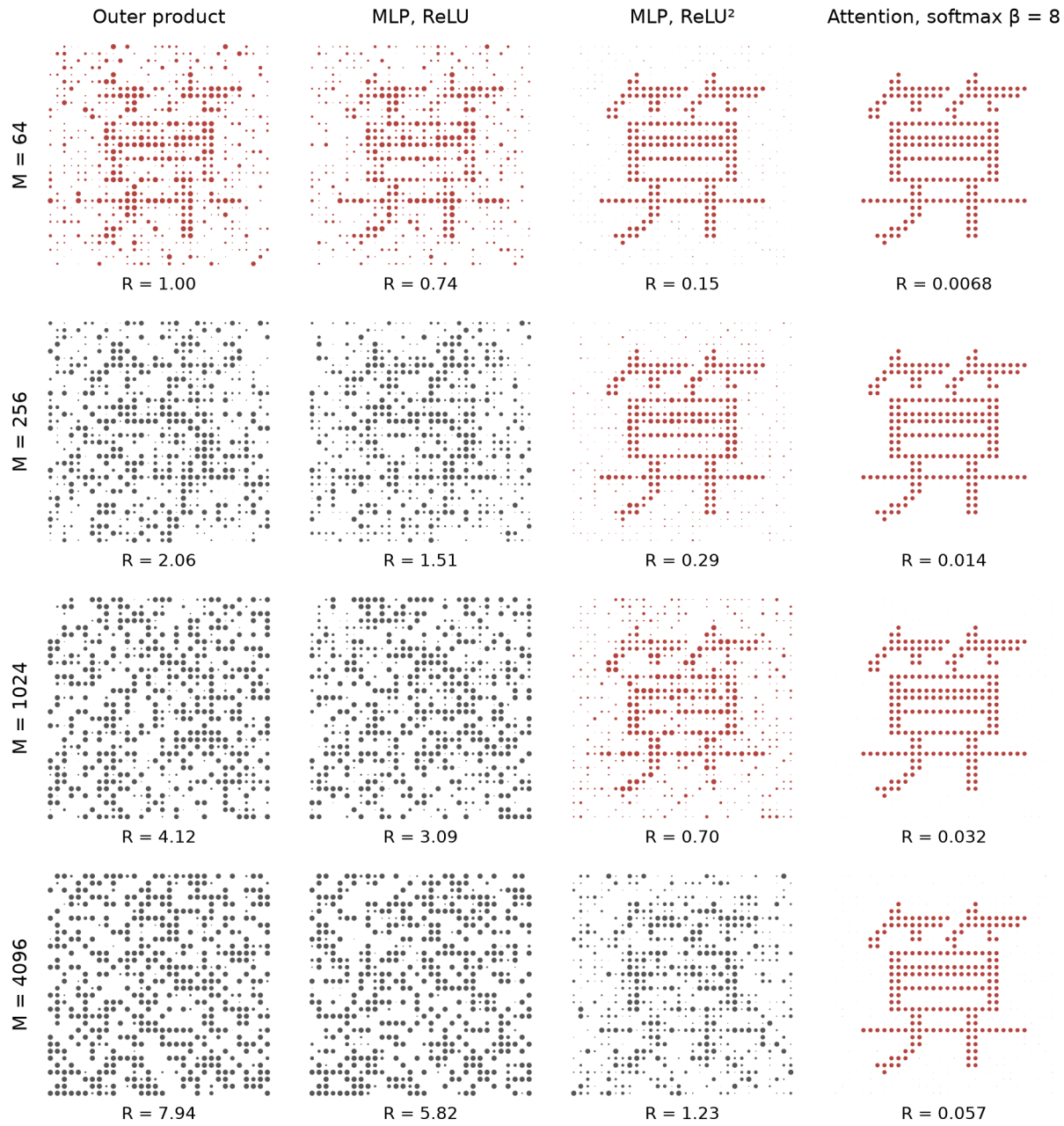


Figure 12: The reference stores recalling the picture of ⊠ (one draw). Rows: the number of memories M; columns: the store. Dot area grows with the recalled pixel; red marks a recall with $R < 1$.

(2) **Attention recalls at every distance.**

> `Plot recall accuracy against query-to-memory distance, and the attention weights of one test sequence.`

Accuracy must exceed 0.95 at every distance and be flat in distance; the reference reaches 1.00. Attention compares the query with every earlier position, so the distance does not matter. In the weights, the query's row puts most of its weight on the memory with the queried key, wherever it lies. A layer stopped too early instead attends to the query itself for a few memory tokens it has not learned to match, and its accuracy sits near 0.95.

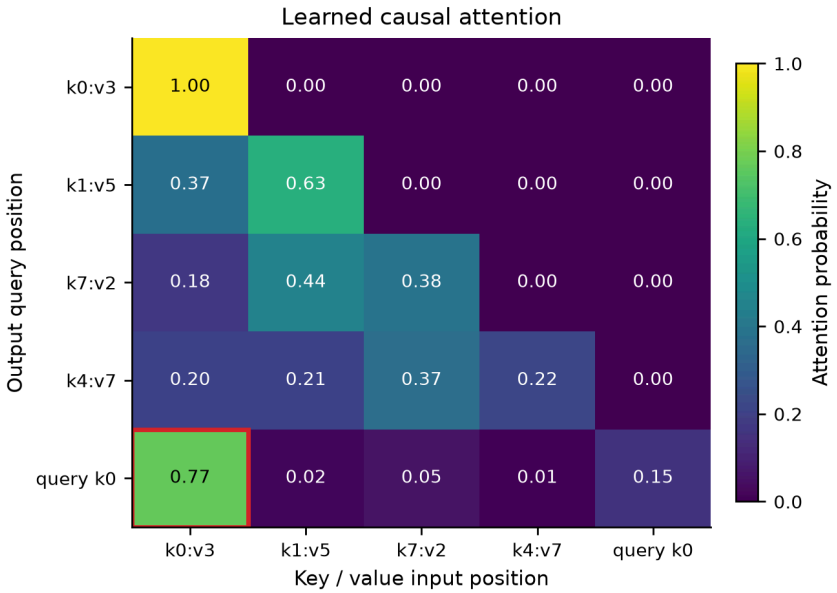


Figure 13: Attention weights of the reference layer on one test sequence. Rows are output positions, columns the positions they read. The query (bottom row) puts 0.77 of its weight on the queried memory (red border).

Part 5: The transformer: training and generating

Goal: assemble the MLP and attention into a transformer. Train a two-layer transformer on TinyStories that beats every count model of Part 2 in held-out bits per character. Sample from it with temperature, top-k, top-p and min-p, each matching its exact target distribution, and show that its own samples are exactly as surprising as it predicts.

1 DO

```
Implement a pre-norm transformer decoder in JAX. Add learned position vectors to the
token embeddings. Each block applies layer norm, multi-head causal attention and a
residual, then layer norm, a GELU MLP four times wider and a residual. End with layer
norm and a readout. Give every linear map a bias.
```

```
Train the transformer on the BPE training stories: 2 layers, width 96, 4 heads,
context 64. Use 6000 AdamW steps of 32 sequences at learning rate 0.003, with 300
warmup steps and cosine decay to a tenth. Report held-out bits per character.
```

While it trains, write the samplers that will turn its predictions into stories.

```
Generate stories from the trained model with temperature 0.8, top-k 20 and top-p 0.9
at temperature 0.8, and min-p 0.1 at temperature 0.7.
```

2 UNDERSTAND

The transformer block. A transformer [8] stacks identical blocks. Each block lets positions exchange information by attention, then transforms each position separately with a small MLP. The text-generating version, a *decoder*, uses causal attention throughout. Figure 14 traces one position's vector from the embeddings at the bottom to the next-token probabilities at the top; the paragraphs below take its boxes in turn.

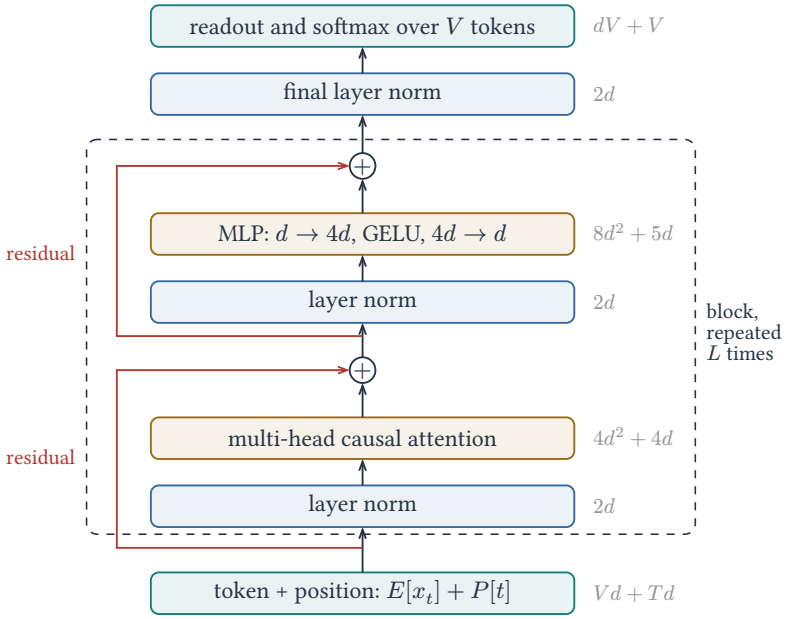


Figure 14: A pre-norm transformer decoder. The dashed block repeats L times; red paths are the residual connections. Grey labels give each box's number of weights, including biases.

Residuals and layer norm. A residual connection adds a sublayer's output to its input, $x \leftarrow x + f(x)$, so each block refines the current vectors and gradients pass through deep stacks. Layer norm rescales each vector to mean zero and variance one across its entries, then applies a learned scale and shift g, b. Applying it before each sublayer, the pre-norm form, keeps training stable:

$$x \leftarrow x + \text{Attention}(\text{LN}_1(x)), \quad x \leftarrow x + \text{MLP}(\text{LN}_2(x)). \tag{21}$$

Multi-head attention. Multi-head attention splits the width d into h heads of width $\frac{d}{h}$, runs attention in each separately, and concatenates the results. Different heads can look for different things. Each head divides its scores by the square root of its own width.

The causal mask. A next-token model must not see the future. Causal attention sets $a_{tj} = 0$ for every $j > t$, by excluding those positions from the sum (Figure 11, right).

The MLP sublayer. The MLP widens each vector to $4d$ entries, applies GELU, a smooth version of $\max(0, u)$, and maps back to d entries. It is the memory store of Part 3, with $4d$ memories.

Position vectors. Part 4 showed that attention ignores order. Text needs order: "dog bites man" differs from "man bites dog". Adding a learned position vector $P[t]$ to the token embedding at position t makes the same token at different positions look different.

Training on every position. A training example is a run of 65 consecutive tokens. The model reads the first 64 and predicts, at every position, the next token. Because attention is causal, each position's prediction uses only earlier tokens, so one forward pass trains 64 predictions. The loss is their average cross-entropy.

Loss at each position. The average over the 64 positions hides how much context helps. Position t of a window has read t tokens, so the mean loss at position t over many windows shows what the model gains from each further token it reads. Kaplan and coauthors [12, Fig. 20] plotted this curve for large models and found that it keeps falling across the whole context. A bigram reads 1 token and a trigram 2, so their loss is the same at every position.

The learning-rate schedule. As in Part 3, warmup raises the learning rate from zero, here linearly over the first 300 steps, and cosine decay then lowers it, here to a tenth of its peak by the last step.

Generating text. Generation is ancestral sampling, as in Part 2: compute the next-token distribution from the tokens so far, draw one token, append it, repeat. The model returns logits z_v. A sampler decides how to draw from them.

Temperature. Sampling with temperature T draws from

$$p_T(v) = \frac{\exp(z_v/T)}{\sum_u \exp(z_u/T)}. \tag{22}$$

$T = 1$ is the model's own distribution. Lower T sharpens it, and as $T \to 0$ it always returns the most likely token, called *greedy* decoding. Higher T flattens it toward uniform.

Truncation samplers. Each rare token is unlikely, but there are hundreds of them, so one of them is drawn often, and they produce most nonsense. Three samplers drop them and renormalize the rest to sum to one. Top-k keeps the k most likely tokens. Top-p, also called nucleus sampling, keeps the most likely tokens until their total probability reaches the threshold. Min-p [13] keeps every token whose probability is at least a fixed fraction of the largest. Figure 15 applies each rule to one distribution. The claim that min-p gives better text than top-p is disputed; here we check only that each sampler draws from its stated distribution.

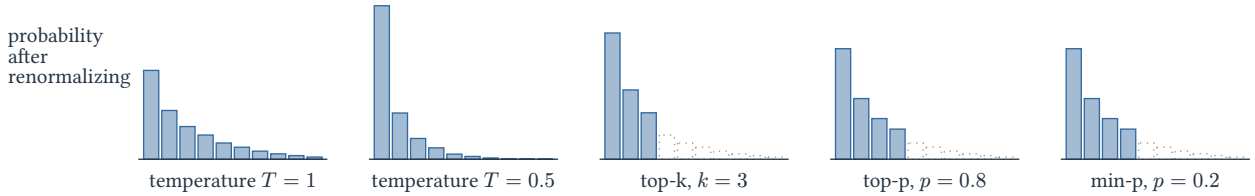


Figure 15: One next-token distribution under each sampler. Kept tokens (filled) are renormalized to sum to one; dropped tokens are dotted. Lowering the temperature sharpens the distribution without dropping any token.

Surprise and entropy of the model's own samples. The surprise of a token is $-\ln p(v)$. Its average under the model's own distribution is, by definition, the entropy $H = -\sum_v p(v) \ln p(v)$. So for text sampled at $T = 1$, the mean surprise the model assigns equals its mean predicted entropy, up to random error. Text sampled at a lower temperature is less surprising than the model predicts. Dividing the difference by its standard error over independent sequences gives a z-score, whose size beyond about 4 signals a real mismatch.

KEY CONCEPTS

- **Block:** pre-norm attention and MLP sublayers, each wrapped in a residual.
- **Multi-head attention:** several narrower attentions side by side.
- **Causal mask:** weights on later positions are exactly zero.
- **Position vectors:** they make the same token at different positions look different.
- **Training:** causal attention lets one pass train a prediction at every position.
- **Schedule:** warmup, then cosine decay of the learning rate.
- **Temperature:** divides logits by T; $T \to 0$ is greedy.
- **Top-k, top-p, min-p:** drop unlikely tokens, then renormalize.
- **Self-consistency:** at $T = 1$, mean surprise equals mean entropy.

3 VERIFY

(1) **The transformer beats every count model.**

> Plot held-out bits per character of the uniform, unigram, bigram and trigram models and the transformer on one chart. Sample a 60-token story opening from each model at temperature 1, with the same seed, and show them in order of bits per character.

Your transformer must lie below the best count model, the 4-gram of Part 2 at 1.315; the reference reaches 1.158 in 81 seconds of training.

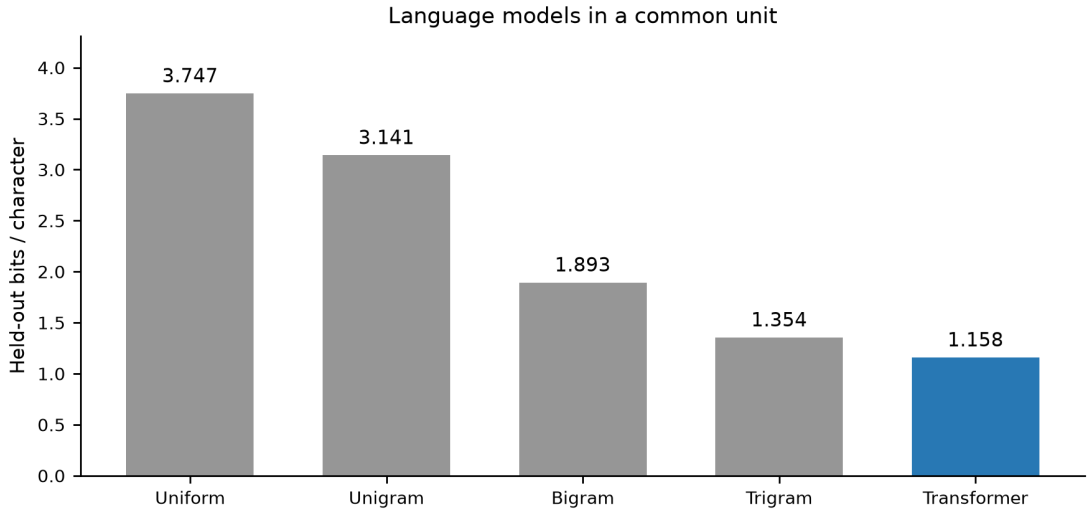


Figure 16: Held-out bits per character of the reference count models and transformer, on the same validation stories and tokens.

The samples make the numbers concrete. Shannon [14] built the same ladder in 1948 from letter and word counts: each step up in the model makes the samples read more like English. Uniform tokens give byte noise; single-token counts give letter soup; pairs and triples of tokens give words and short phrases that do not join up; the transformer keeps a sentence together. In the reference:

Model	Bits / char	Sample, temperature 1
Uniform	3.75	`□□f□0□□ □□9g□U□I□□□□□□V□□□HS□□\□□□z□S□Bo□□s□□w*□;<□zX4□3I□`
Unigram	3.14	`igo have on. l A a the u soc al..ameore. a Timp play sa hyT. notr."g she hers so w f had toeds lieht. were p Theyl happy toenic the were named ".ri`
Bigram	1.89	`Once upon a smice on the good at her coiles. He said, Tim. She eld him. They home took. Tim. She did noticed their put the mapen to ke bothy can leade`
Trigram	1.35	`u don't worm!"/The nose--hearthy./They look together to soar. He got to the bat. The carty. The pond, there was a chank you for l`
Transformer	1.16	`Once upon a time there was a little rabbit. Everyday loved to ride under his grass to take turns to shoe the pink car in the kites. He wanted to carr`

Table 1: One reference sample per model, seed 1848, end-of-story token excluded. A slash marks a line break and a square a byte that is not a whole character.

(2) **The transformer uses its whole context.**

> Score the held-out stories in windows of 64 tokens. Plot the transformer's mean loss at each position against the number of tokens read, with the bigram's and trigram's mean loss as lines.

After reading 32 or more tokens, the transformer's loss must lie below the trigram's 2.25 nats per token; the reference gives 1.85 over the last 8 positions. With one token read it is worse than the bigram, which counts every pair of tokens in the training stories exactly. It passes the trigram after 4 tokens and keeps improving to 64. This is where its lead in bits per character comes from: the trigram stops reading after 2 tokens.

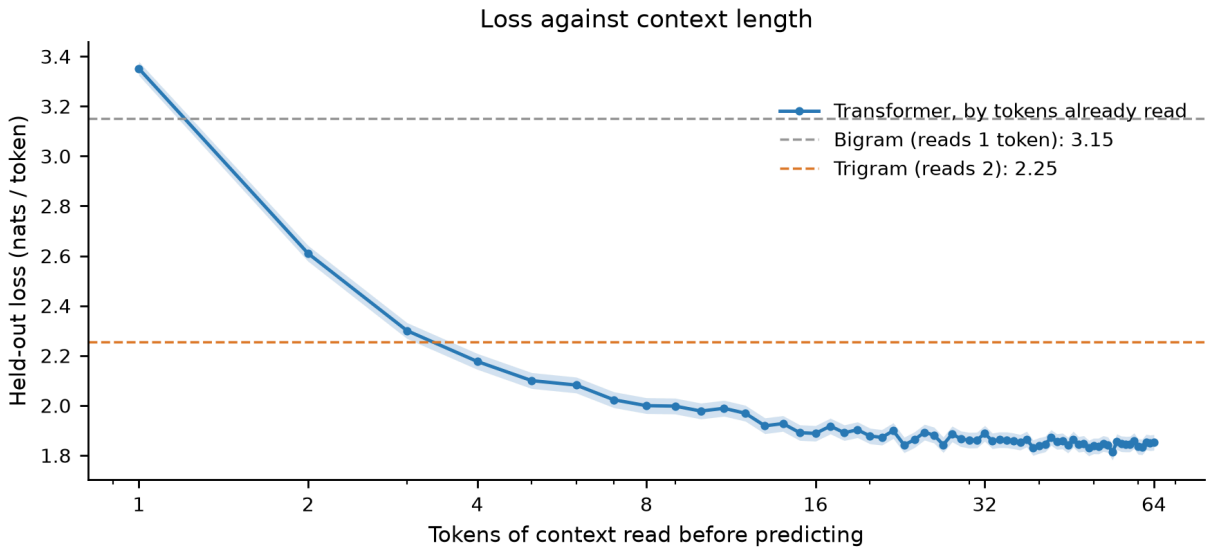


Figure 17: Reference transformer's held-out loss at each position of 64-token windows, with a band of two standard errors. Dashed lines are the average loss of two earlier models.

(3) **Each sampler draws from its target distribution.**

> From one fixed context, draw 100000 next tokens with each sampler, using the settings above. Compare the counts with each sampler's exact distribution by a chi-squared test. Check that temperature 0.0001 always returns the most likely token.

Every p-value must exceed 0.001; the reference gives values between 0.15 and 0.96. Temperature 0.0001 must return the most likely token every time. A top-p sampler that drops the token crossing the threshold gives $p \approx 0$.

(4) **The model's samples are as surprising as it predicts.**

> Generate 128 sequences of 128 tokens at temperature 1. Compare their mean surprise with the model's mean predicted entropy, as a z-score over sequences. Repeat with sequences sampled at temperature 0.7, scored by the temperature-1 model.

At $T = 1$ the z-score must lie between -4 and 4; the reference gives 1.0. At $T = 0.7$ it must exceed 40 in size; the reference gives -71.

Extension: where the attention heads look in a story. Each of the 8 heads of your transformer spreads every token's attention over the tokens before it. Draw these weights for the opening of a held-out story, one heat map per head. In the reference, the heads of the first layer put much of their weight on the last few tokens, while the heads of the second layer reach further back. In the right-hand map, "park" in "went to the park" reads "went", and "bug" in "She saw a tiny bug" reads "saw": the word that predicts what comes next is not always the nearest one. The maps show where a head reads, not what it computes; they are a picture, not an explanation.

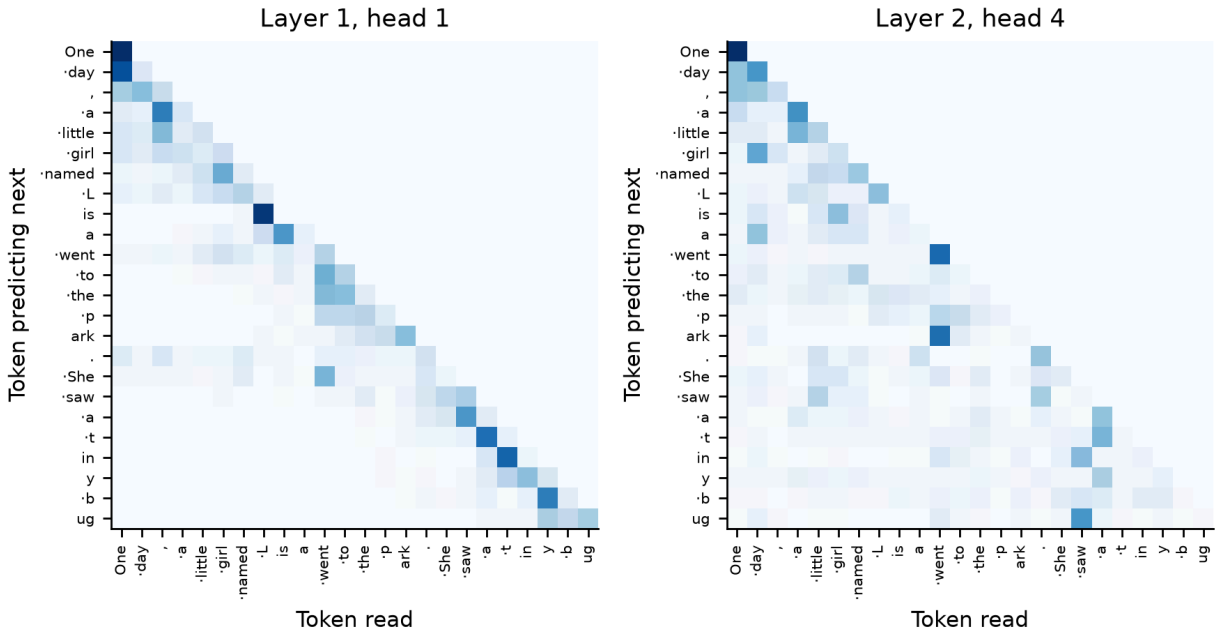


Figure 18: Attention weights of two heads of the reference transformer. Row t is the token predicting the next one; its weights over the tokens read sum to one. A middle dot marks a space.

Part 6: Scaling laws: how to spend a compute budget

Goal: refit the Chinchilla scaling law $L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$ to published training runs of models from 57 million to 16 billion parameters. Recover the replication's exponents $\alpha \approx 0.35$ and $\beta \approx 0.37$, show that the parameters printed in the original paper miss those runs, and derive the compute-optimal split: about 20 training tokens per parameter at Chinchilla's budget.

1 DO

Download the training runs:

```
curl -L -o week8/data/chinchilla-runs.csv \
  https://raw.githubusercontent.com/epoch-research/analyzing-chinchilla/92258837425e1b5f2851d624287f0120583a3d0e/data/svg_extracted_data.csv
```

```
Read the Chinchilla training runs in week8/data/: model size N, training compute C in FLOP, and final loss L. Compute the training tokens D = C / (6N). Drop the five runs of highest loss. Fit the scaling law L = E + A / N^alpha + B / D^beta by the Huber loss of log L, with delta = 0.001. Start L-BFGS from a grid of values and keep the best fit.
```

```
Refit on 400 bootstrap resamples of the runs, starting from the best fit. Report each parameter's standard error.
```

```
Minimize the fitted law at fixed training compute C = 6ND. Report the compute-optimal model size N* and tokens D* for C from 1e18 to 1e25 FLOP, and D*/N* at C = 5.76e23 with a 95% bootstrap interval.
```

2 UNDERSTAND

Size and data. Part 5 trained one model of one size on one budget of tokens. Kaplan and coauthors [12] found that the loss of language models falls as a power of their size, and also as a power of the number of training tokens, over many orders of magnitude. A scaling law with both resources is

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}. \tag{23}$$

N counts the parameters and D the training tokens. E is the loss an infinitely large model would reach on unlimited data: text is never perfectly predictable. The second term is what a model of N parameters cannot represent, and the third what D tokens cannot teach. Each term falls as a power, a straight line of slope $-\alpha$ or $-\beta$ on log-log axes.

Training compute. Each training token costs about $2N$ multiplications and additions in the forward pass and $4N$ in the backward pass. The training compute of a run is therefore $C \approx 6ND$ floating-point operations, FLOP. A fixed budget can buy a large model trained on few tokens or a small model trained on many.

The Chinchilla runs. Hoffmann and coauthors [15] trained over 400 transformers to find the best split. The downloaded file holds 245 of them, models of 57 million to 16 billion parameters trained on 0.8 to 320 billion tokens. Besiroglu and coauthors [16] read the points off Figure 4 of the paper. Each row gives N, C and the final loss in nats per token. The five runs of highest loss lie far above the others, so the replication leaves them out.

Fitting with the Huber loss. The losses run from 2 to 5. Fitting $\log L$ instead of L weighs a 1% miss the same at every loss. The Huber loss of a residual r is $\frac{r^2}{2}$ for $|r| \le \delta$ and grows only linearly beyond, so a few runs that miss by much pull the fit less than in least squares. With $\delta = 0.001$, nearly every run is in the linear part. The law is not a convex function of its five parameters, and L-BFGS, a gradient method, can stop in a local minimum. Starting it from a grid of values and keeping the lowest result finds the best fit.

Bootstrap errors. The runs are one sample of what could have been trained. A bootstrap resample draws 240 runs with replacement from the 240 fitted ones. Refitting each resample gives a spread of parameters; its standard deviation estimates each parameter's standard error, and the middle 95% of any derived number gives its interval.

The compute-optimal split. For a budget C, the compute-optimal model minimizes L along $ND = \frac{C}{6}$. Setting the derivative to zero gives

$$N^* = G\left(\frac{C}{6}\right)^{\frac{\beta}{\alpha+\beta}}, \quad D^* = \frac{1}{G}\left(\frac{C}{6}\right)^{\frac{\alpha}{\alpha+\beta}}, \quad G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}}. \tag{24}$$

With $\alpha \approx \beta$, both grow like $\sqrt{C}$: ten times the compute buys about three times the parameters and three times the tokens. Kaplan and coauthors had advised growing the model much faster than the data. Chinchilla, 70 billion parameters trained on 1.4 trillion tokens, 20 tokens per parameter, beat the 280-billion-parameter Gopher trained on 300 billion tokens with about the same compute.

The replication. Hoffmann and coauthors printed $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$ and $\beta = 0.28$ for this law. Besiroglu and coauthors found that these numbers do not fit the runs in the paper's own figure, and that they imply far more tokens per parameter than the paper's other two methods and the model it trained. Their refit gives $\alpha = 0.35$ and $\beta = 0.37$. A published number is evidence only if it can be recomputed.

KEY CONCEPTS

- **Scaling law:** $L = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$; a floor plus one power law in size and one in data.
- **Training compute:** $C \approx 6ND$ FLOP; a budget trades size against data.
- **Huber loss on log L:** relative misses, with outliers pulling linearly; a grid of starts avoids local minima.
- **Bootstrap:** refit on resampled runs; the spread is the standard error.
- **Compute-optimal split:** N^* and D^* both grow like $C^{\frac{1}{2}}$; about 20 tokens per parameter.

3 VERIFY

(1) **The refit recovers the replication's exponents.**

```
Plot the final loss against model size, with the fitted law at 1e19, 1e20 and 1e21
FLOP, and the measured loss against the fitted loss.
```

The exponents must lie in $0.32 \le \alpha \le 0.38$ and $0.32 \le \beta \le 0.42$, and the floor in $1.75 \le E \le 1.90$. The reference gives $\alpha = 0.347 \pm 0.014$, $\beta = 0.367 \pm 0.021$, $E = 1.817 \pm 0.025$, $A = 478$ and $B = 2,143$, as in the replication. At fixed compute the loss has a minimum in N: too small a model cannot represent the text, too large a one sees too few tokens. The measured losses sit on the diagonal.

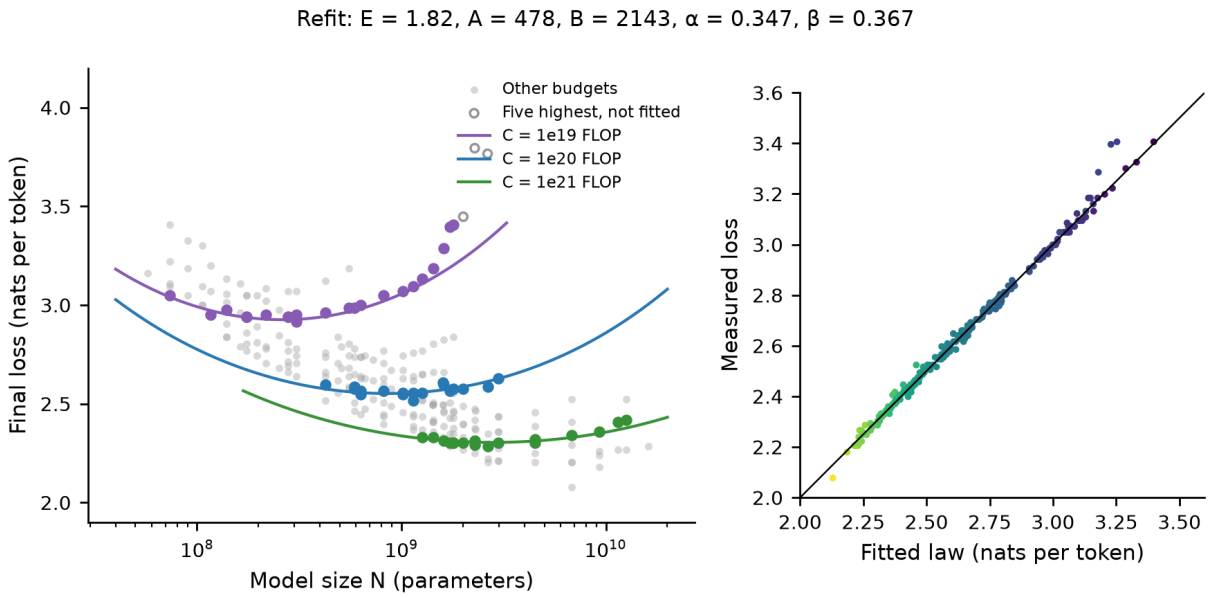


Figure 19: Left: the Chinchilla runs, with those within 15% of three budgets coloured and the reference law at that compute. Right: measured against fitted loss of the 240 fitted runs, coloured by compute.

(2) **The printed parameters miss the runs.**

> `Plot the residuals of log loss against model size for your fit and for the printed parameters E = 1.69, A = 406.4, B = 410.7, alpha = 0.34, beta = 0.28. Report each mean residual.`

Your mean residual must lie within ± 0.005; the reference gives -0.0007. The printed parameters must give a mean above 0.01; the reference gives $+0.017$. They predict a loss about 1.7% too high for almost every run, at every size. Their $\beta = 0.28$ lies four standard errors below the refit's 0.367.

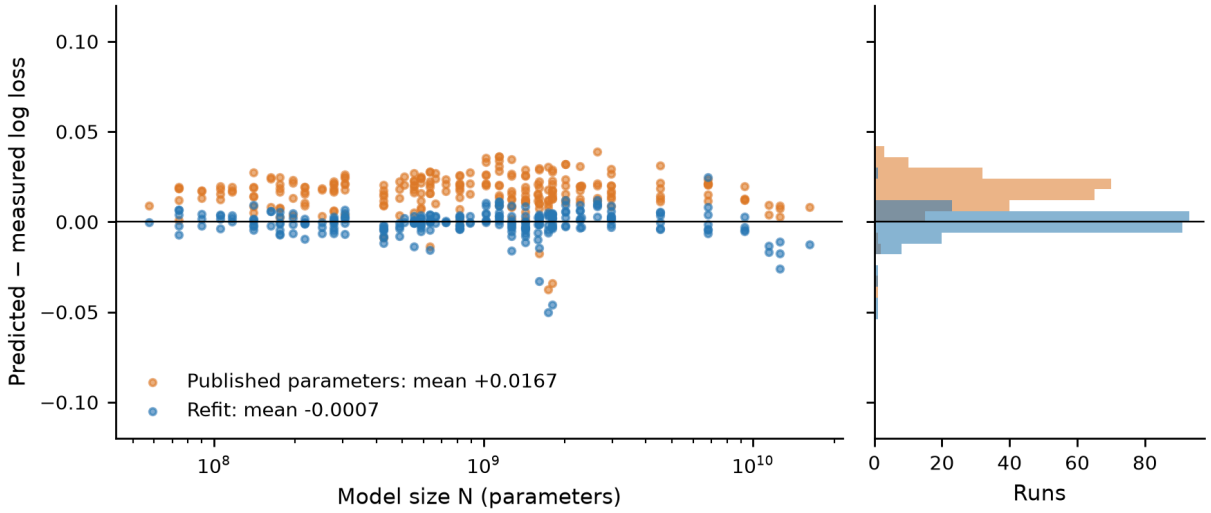


Figure 20: Predicted minus measured log loss for the reference refit (blue) and the printed parameters (orange), against model size and as histograms.

(3) **About 20 tokens per parameter at Chinchilla's budget.**

> `Plot N* and D* against compute on log-log axes for your fit and the printed parameters. Mark Chinchilla: 70 billion parameters and 1.4 trillion tokens.`

At $C = 5.76 \times 10^{23}$ FLOP, $\frac{D^*}{N^*}$ must lie between 10 and 30; the reference gives 17.9, with a 95% interval of 7.7 to 37 that holds Chinchilla's 20. The model size grows like $C^{0.51}$ and the data like $C^{0.49}$. The printed parameters

give 93 tokens per parameter, a model a third of Chinchilla's size: the law the paper printed does not support the model the paper built.

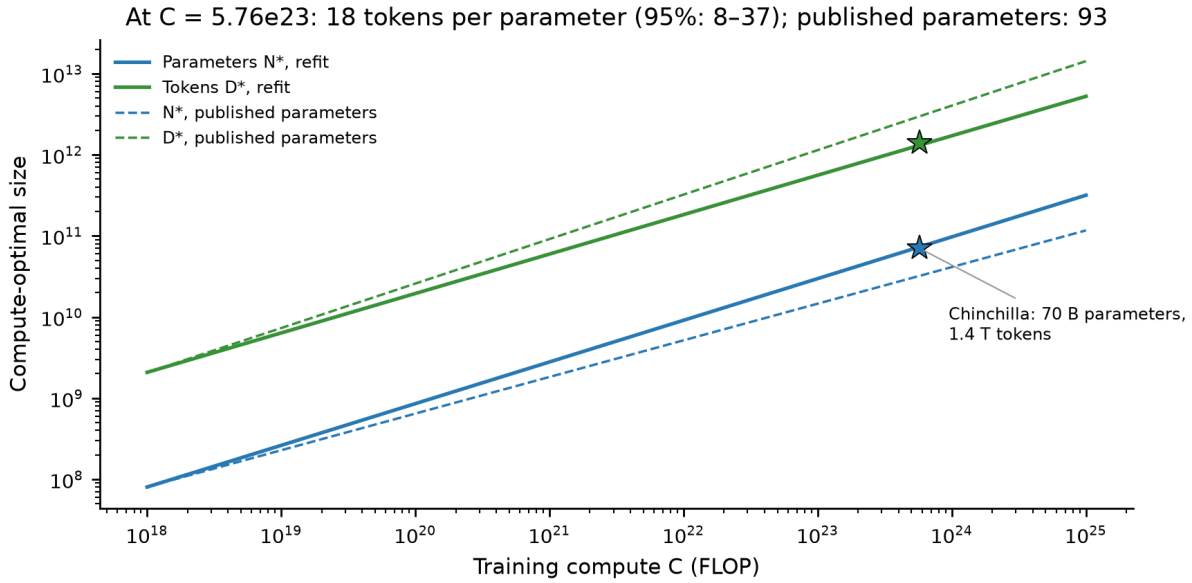


Figure 21: Compute-optimal model size and training tokens of the reference refit (solid) and the printed parameters (dashed). Stars: Chinchilla.

Save the results and prepare to show your work

> Update README.md with each evidence figure beside its producing command. Explain the tokenizer, the recalled pictures, the model comparison in bits per character and the scaling fit. Record the corpus download address. Keep the corpus out of git. Push the source and evidence.

Show the completed evidence at the start of Week 9. Nothing is submitted.

[1] S. Tulkens and T. van Dongen, "Model2Vec: Fast State-of-the-Art Static Embeddings." [Online]. Available: https://github.com/MinishLab/model2vec

[2] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and their Compositionality," in *Advances in Neural Information Processing Systems*, 2013.

[3] R. Eldan and Y. Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?," *arXiv preprint arXiv:2305.07759*, 2023, [Online]. Available: https://arxiv.org/abs/2305.07759

[4] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer Feed-Forward Layers Are Key-Value Memories," in *Empirical Methods in Natural Language Processing*, 2021. [Online]. Available: https://arxiv.org/abs/2012.14913

[5] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A Neural Probabilistic Language Model," *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003, [Online]. Available: https://www.jmlr.org/papers/v3/bengio03a.html

[6] N. Cammarata, "Neural Networks Are Already Clean." [Online]. Available: https://www.goodfire.com/research/neural-networks-are-already-clean

[7] A. Wang, "You Could Have Invented Transformers from Memory." [Online]. Available: https://alexwang.ai/posts/associative-memory/

[8] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[9] D. Krotov and J. J. Hopfield, "Dense Associative Memory for Pattern Recognition," in *Advances in Neural Information Processing Systems*, 2016. [Online]. Available: https://arxiv.org/abs/1606.01164

[10] M. Demircigil, J. Heusel, M. Löwe, S. Upgang, and F. Vermet, "On a Model of Associative Memory with Huge Storage Capacity," *Journal of Statistical Physics*, vol. 168, pp. 288–299, 2017, [Online]. Available: https://arxiv.org/abs/1702.01929

[11] H. Ramsauer *et al.*, "Hopfield Networks is All You Need," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://arxiv.org/abs/2008.02217

[12] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020, [Online]. Available: https://arxiv.org/abs/2001.08361

[13] M. Nguyen, A. Baker, C. Neo, A. Roush, A. Kirsch, and R. Shwartz-Ziv, "Turning Up the Heat: Min-p Sampling for Creative and Coherent LLM Outputs," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://arxiv.org/abs/2407.01082

[14] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[15] J. Hoffmann, S. Borgeaud, A. Mensch, and others, "Training Compute-Optimal Large Language Models," in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: https://arxiv.org/abs/2203.15556

[16] T. Besiroglu, E. Erdil, M. Barnett, and J. You, "Chinchilla Scaling: A Replication Attempt." [Online]. Available: https://arxiv.org/abs/2404.10102